The National University of Lesotho

Department of Mathematics and Computer Science

Faculty of Science and Technology

CS4430: Distributed Database Systems

BorderFlow

Final Report: Kubernetes Deployment & Prototype Demonstration

Coordinator: Mr. Khobatha Setetemela

Due: April 2026

https://github.com/karaboGrace/borderflow

Student Number	Name
200900706	Moleleki H.G.
202201934	Nkolokoane L.N.
202201176	Mohlapisi B.I.
202201173	Ramochele M.B.
202200990	Motaung M.
202201992	Makoetje K.G.M.
202201955	Leiee L.J.
202201074	Ramainoane M.A.R.

1. Introduction

This final report documents the complete implementation and Kubernetes deployment of BorderFlow, a distributed offline-first container logistics tracking system developed for CS4430: Distributed Database Systems. The system models cross-border container logistics across four geographically dispersed sites, demonstrating correctness under network failures and partition recovery.

The implementation fulfils all Week 6 deliverables as specified in the project handout:

- Kubernetes deployment with manifests for all services
- Seed dataset and initialization scripts
- Working prototype demonstrating the complete consignment workflow
- Failure and recovery demonstration using network partition simulation
- Final report documenting architecture, implementation, and findings

2. System Overview

2.1 Architecture Summary

BorderFlow implements a multi-master replication model with a central synchronization hub. Five services are deployed independently:

Service	Role	Database	Port
Origin Depot (Maseru)	Container registration, driver assignment, dispatch	SQLite (local)	8001 / 30001
Border Post (Maseru Bridge)	Customs clearance, handovers, status updates	SQLite (local)	8002 / 30002
Port Site (Durban)	Vessel loading, port milestones	SQLite (local)	8003 / 30003
Destination Hub (Johannesburg)	Delivery confirmation, gate-in	SQLite (local)	8004 / 30004
Control Tower	Global sync hub, conflict resolution, dashboard	SQLite / PostgreSQL	8000 / 30000

2.2 Key Design Principles

- Offline-First Operation: Each site stores data locally in SQLite and queues changes for synchronization. Sites continue accepting writes during network partitions.
- Multi-Master Replication: All sites accept writes independently, prioritizing availability over strong consistency (AP from CAP theorem).
- Push-Based Synchronization: Sites push pending changes to the Control Tower every 10 seconds when connectivity is available.

- Idempotency: Every operation is assigned a unique operation ID. The Control Tower checks received IDs against a processed_operations log to prevent duplicate processing.
- Last Write Wins Conflict Resolution: Concurrent updates are resolved by comparing version numbers (higher wins) with timestamp as tiebreak.

3. Implementation Details

3.1 Technology Stack

Component	Technology	Justification
Site Service API	Python / FastAPI	Lightweight, async, automatic OpenAPI docs
Edge Database	SQLite	Zero-config, embedded, survives restarts, perfect for edge
Control Tower DB	SQLite / PostgreSQL	ACID transactions for reliable conflict resolution
Containerization	Docker	Reproducible builds, isolated environments
Orchestration	Kubernetes	Production-grade deployment, network policy simulation
Sync Protocol	REST / JSON over HTTP	Platform-agnostic, works across network boundaries

3.2 Site Service

Each of the four edge sites runs an identical FastAPI application configured via environment variables (SITE_ID, SITE_NAME, CONTROL_TOWER_URL). The service exposes the following REST endpoints:

Endpoint	Method	Description
POST /containers	POST	Register a new container at the site
GET /containers	GET	List all containers known to this site
POST /containers/status	POST	Update container status (queues sync)
POST /trips	POST	Assign a driver/vehicle to a container
POST /milestones	POST	Record a milestone event
POST /incidents	POST	Report an incident or exception
GET /sync/status	GET	View pending sync queue depth

Every write operation records the change in a local sync_queue table with a unique operation_id. A background async task runs every 10 seconds, batches pending operations, and POSTs them to the Control Tower's /sync/receive endpoint.

3.3 Control Tower Sync Service

The Control Tower receives batched sync operations from all sites and applies them to the global database using the following logic:

- Idempotency Check: If the operation_id already exists in processed_operations, the operation is skipped without error.
- Conflict Detection: For container updates, the incoming version number is compared with the existing record. If the incoming version is higher, the update is applied (Last Write Wins). If equal, timestamps are compared. If the existing record is newer, the incoming update is skipped.
- Acknowledgement: The Control Tower responds with counts of accepted, skipped, and conflict-resolved operations. The site then clears its sync queue.

4. Kubernetes Deployment

4.1 Deployment Architecture

All components are containerized using Docker and deployed on a Kubernetes cluster. The following manifests are included in the k8s/ directory of the repository:

Manifest File	Contents
k8s/postgres.yaml	PostgreSQL Deployment, PersistentVolumeClaim, Service
k8s/control-tower.yaml	Control Tower Deployment, NodePort Service (port 30000)
k8s/sites.yaml	Four site Deployments, PersistentVolumeClaims, NodePort Services (30001-30004)
k8s/network-partition.yaml	NetworkPolicy to simulate Border Post going offline

4.2 Deployment Steps

The following commands reproduce the full deployment from scratch:

```
# Build Docker images
docker build -t borderflow-site:latest ./site_service
docker build -t borderflow-control-tower:latest ./control_tower
# Deploy to Kubernetes
kubectl apply -f k8s/postgres.yaml
kubectl apply -f k8s/control-tower.yaml
kubectl apply -f k8s/sites.yaml
# Load seed data
python scripts/seed.py
```

4.3 Persistent Storage

Each site service uses a Kubernetes PersistentVolumeClaim (500Mi) mounted at /data to ensure SQLite databases survive pod restarts. The Control Tower uses a 1Gi PVC for PostgreSQL data. This guarantees that locally queued operations are not lost during container restarts, which is essential for the offline-first guarantee.

5. Failure and Recovery Demonstration

5.1 Scenario: Border Post Network Partition

The demonstration follows this sequence to prove correctness under partial failure:

Step	Action	Expected Behaviour
1	All services started, seed data loaded	Dashboard shows containers from all sites
2	Border Post network partition applied (kubectl apply -f k8s/network-partition.yaml)	Border loses connectivity to Control Tower
3	New container registered and cleared at Border while offline	Data saved locally; sync queue grows
4	Milestone and status update recorded at Border (CLEARED)	Changes queued, not yet in Control Tower
5	Control Tower queried — Border changes absent	Confirms partition is working
6	Network partition removed (kubectl delete -f k8s/network-partition.yaml)	Connectivity restored
7	Wait 10-15 seconds for sync worker to fire	Border pushes all queued operations
8	Control Tower queried again	Border changes now visible; sync log updated

5.2 Conflict Resolution Test

To demonstrate Last Write Wins conflict resolution:

- A container is registered at the Depot and synced to Control Tower (version=1).
- The Border Post updates the same container to IN_TRANSIT (version=2) while offline.
- The Depot simultaneously updates the container to AT_BORDER (version=2) with a later timestamp.
- When both sync, the Control Tower detects the version conflict. The Depot update wins because its timestamp is later.
- The sync log records conflicts_detected=1 for this batch.

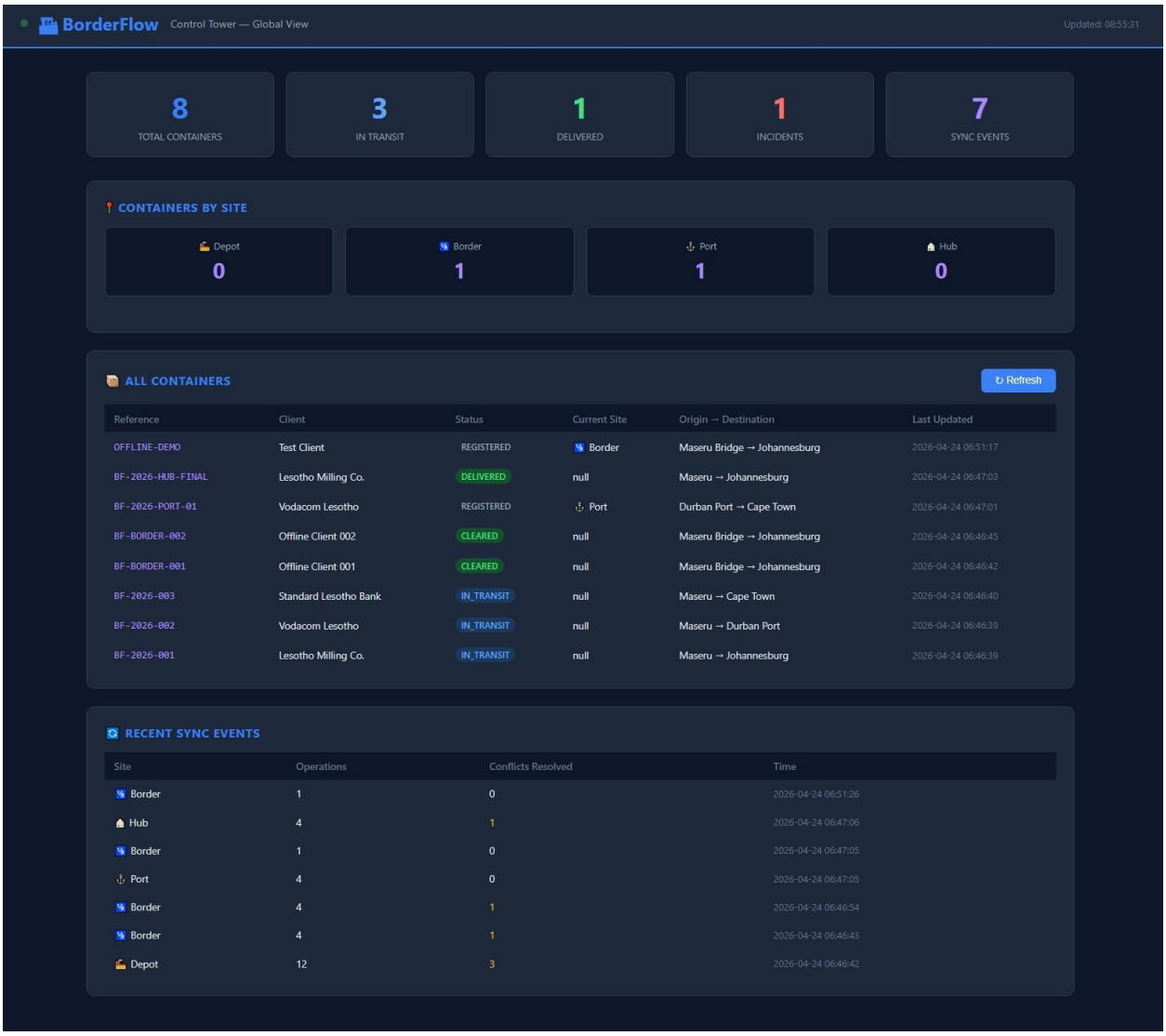
5.3 Idempotency Test

To demonstrate duplicate protection, the same sync batch is submitted to /sync/receive twice. The second submission returns skipped_duplicates equal to the batch size and accepted=0, confirming that all operation_ids were found in the processed_operations table and no data was double-written.

6. Complete Consignment Workflow

The seed script (scripts/seed.py) demonstrates one complete end-to-end workflow as required by the handout:

Step	Site	Action	Status After
1	Depot	Container BF-2026-001 registered for Lesotho Milling Co.	REGISTERED
2	Depot	Driver Thabo Mokoena assigned, vehicle LSO-1234-AA	REGISTERED
3	Depot	Container dispatched	IN_TRANSIT
4	Border	Container cleared by customs (offline)	CLEARED
5	Border	Document problem incident reported	CLEARED
6	Port	Vessel loading milestones recorded	AT_PORT
7	Hub	Delivery confirmed, seal verified	DELIVERED
8	All	Sync fires — Control Tower updated	Global view complete



7. Correctness Argument

7.1 Safety Properties

- No data loss: Every write is persisted to local SQLite before returning a response. The sync queue is also stored locally and survives process restarts.
- No duplicate processing: Idempotency keys (operation_id UUIDs) guarantee that even if a site retries a sync batch, the Control Tower applies each operation at most once.
- Conflict resolution is deterministic: Last Write Wins with version vectors and timestamp tiebreak produces a consistent result regardless of which site's sync arrives first.

7.2 Liveness Properties

- Eventual consistency: As long as a site eventually regains connectivity, all its queued operations will reach the Control Tower. The 10-second sync interval ensures prompt propagation.
- Availability: Sites continue operating fully during partitions. No operation requires cross-site coordination to complete locally.
- Recovery: When connectivity is restored, the sync worker automatically processes the backlog without manual intervention.

8. Limitations and Future Work

- Clock skew: Last Write Wins relies on timestamps. NTP synchronization is assumed but not enforced. True version vectors would provide stronger causal guarantees.
- Semantic conflicts: The system flags DELIVERED vs IN_TRANSIT conflicts in the design but the current implementation uses LWW for all types. Manual resolution UI is out of scope.
- Security: REST endpoints have no authentication. Production deployment would require mTLS between services and JWT-based API authentication.
- Scale: The Control Tower is a single point of failure. A replicated PostgreSQL cluster with leader election would address this in production.

9. Team Contributions

Member	Student No.	Contributions
Moleleki H.G.	200900706	Frontend dashboard UI, Control Tower integration, project coordination
Nkolokoane L.N.	202201934	Frontend UI, seed data script, testing
Mohlapisi B.I.	202201176	Docker containerization, Kubernetes manifests, network policies
Ramochele M.B.	202201173	Kubernetes deployment, infrastructure, Docker Compose
Motaung M.	202200990	Backend APIs, version vectors, sync service design
Makoetje K.G.M.	202201992	Idempotency implementation, sync service, conflict resolution
Leiee L.J.	202201955	Site business logic (depot, border, port, hub services)
Ramainoane M.A.R.	202201074	Site business logic, milestone/incident tracking, documentation

10. References

1. Gilbert, S. and Lynch, N., 2002. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, 33(2), pp.51-59.

2. DeCandia, G. et al., 2007. Dynamo: Amazon's Highly Available Key-value Store. ACM SIGOPS, pp.205-220.

3. Ozsu, M.T. and Valduriez, P., 2020. Principles of Distributed Database Systems. 4th ed. Cham: Springer.

4. Madapparambath, G. and McKendrick, R., 2024. The Kubernetes Bible. 2nd ed. Birmingham: Packt Publishing.

5. Burns, B. and Villalba, E., 2023. Kubernetes Best Practices. 2nd ed. Sebastopol: O'Reilly Media.